

AQUÁRIO INTELIGENTE

Gabrieli Eduarda Lembeck

Heloisa Rebello Cabral

Julio Bezerra De Mattos Manoel

Mileine Da Silva De Freitas

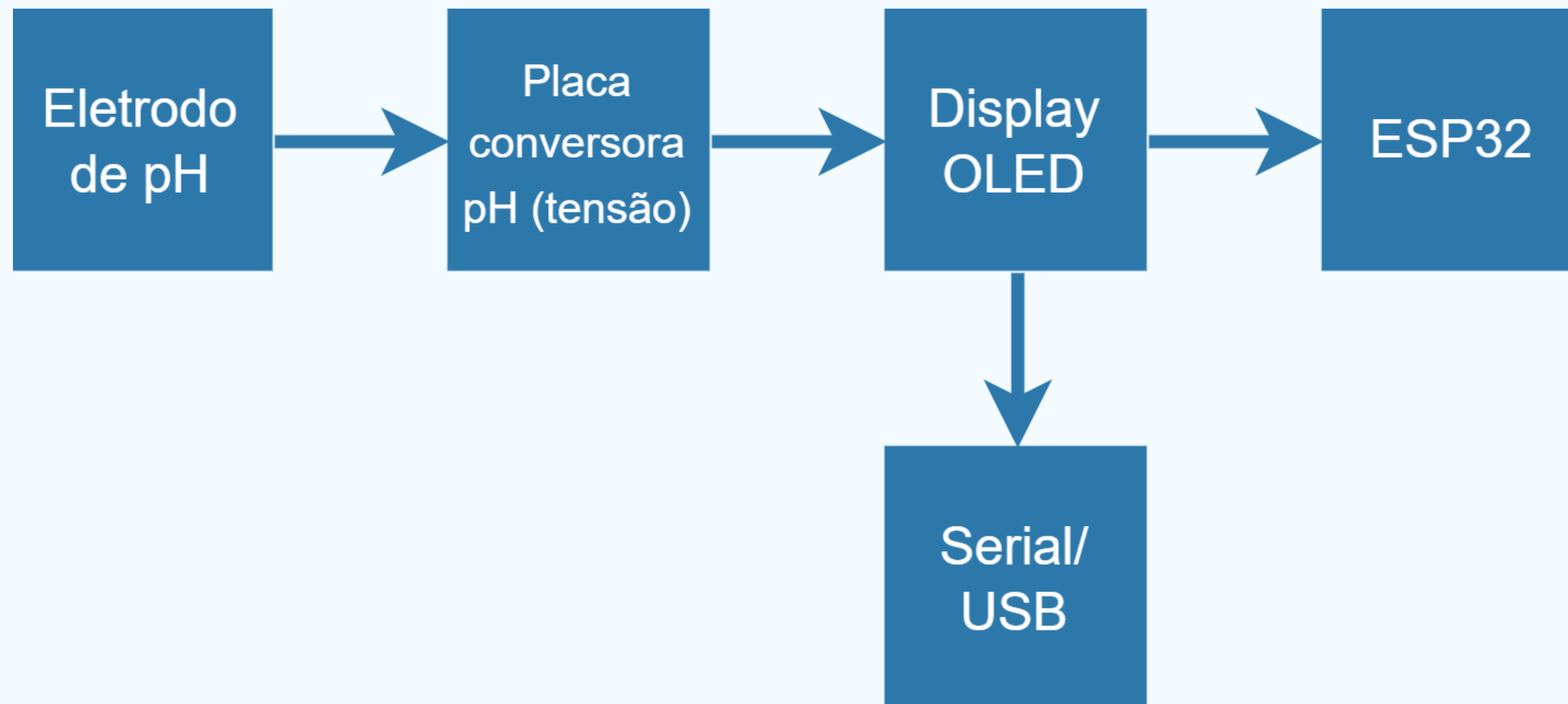
Thomas Henry Steinback

PROBLEMA

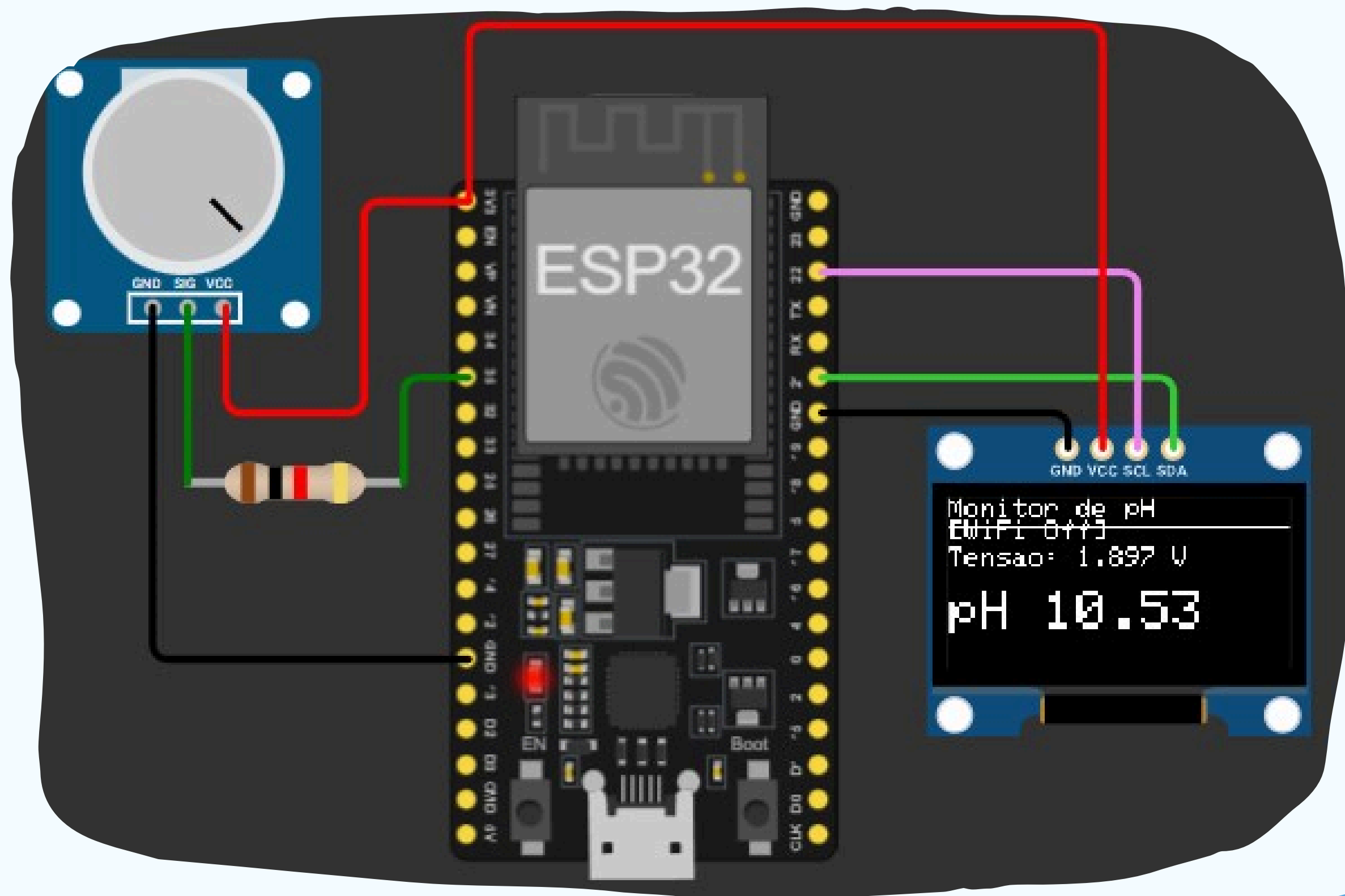
Monitorar constantemente as condições da água de um aquário, **manualmente**, o que pode fazer com que alterações e oscilações no estado e condições da água **não sejam percebidos rapidamente**.

A proposta é integrar diferentes sensores em um único sistema, permitindo o **monitoramento contínuo** dos parâmetros do aquário e o envio dessas informações para um **servidor na nuvem**. Dessa forma, os **dados** podem ser **acompanhados e analisados** de maneira centralizada.

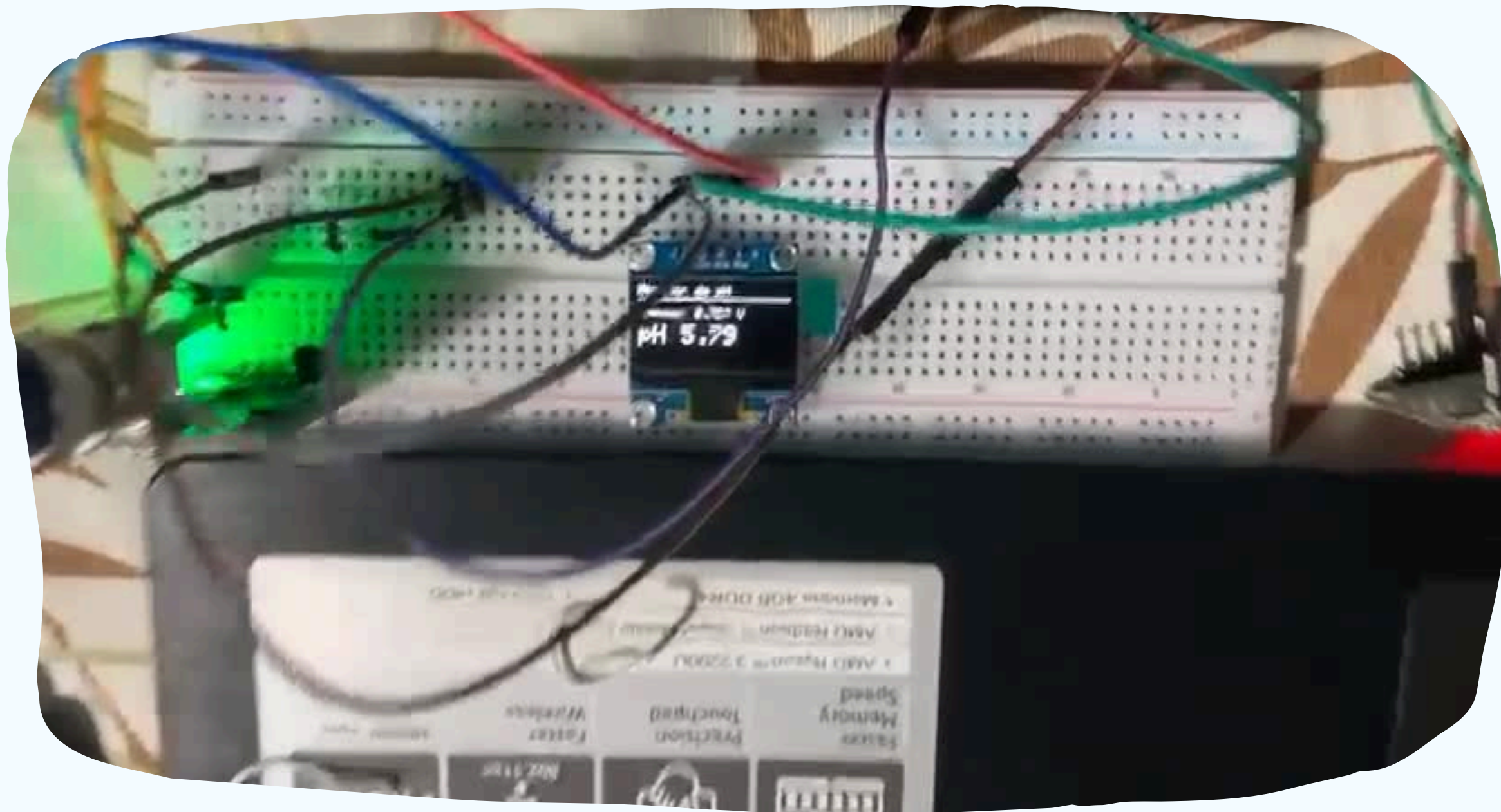
DIAGRAMA EM BLOCOS



PROTÓTIPO



PROTÓTIPO



COMPONENTES

- **ESP32:** microcontrolador central. Faz a leitura do sinal analógico, converte para valor de pH e envia o resultado ao display.
- **Sensor de pH + módulo conversor:** o eletrodo mede a concentração de íons H^+ na água e gera uma tensão proporcional ao pH; o módulo condiciona esse sinal para a faixa de 0–3,3V lida pelo ESP32.
- **Display OLED (I2C, 128x64):** exibe o valor de pH e a tensão lida em tempo real, direto na bancada.
- **Alimentação:** ESP32 fornece 3,3V e GND para o sensor de pH e para o display.
- **Sinal do sensor:** saída analógica do módulo de pH → pino GPIO35 do ESP32 (entrada ADC).
- **Display:** comunicação via barramento I2C (2 fios — SDA e SCL) entre ESP32 e OLED.

FUNCIONAMENTO

1. ESP32 lê a tensão do sensor (analogRead / analogReadMilliVolts).
2. Converte a tensão em valor de pH por uma fórmula linear.
3. Envia o valor para o display OLED.
4. Registra os dados também via Serial (debug em bancada).
5. Repete a cada 200ms.

CÓDIGO

- Bibliotecas e Dependências;
- Definições de Hardware;
- Parâmetros de Calibração do pH

```
1  #include <Arduino.h>
2  #include <Wire.h>
3  #include <Adafruit_GFX.h>
4  #include <Adafruit_SSD1306.h>
5  #include <WiFi.h>
6  #include <PubSubClient.h>
7  #include <HTTPClient.h>
8
9  #define PH_PIN 35
10 #define NUM_SAMPLES 15
11
12 // ===== OLED =====
13 #define SCREEN_WIDTH 128
14 #define SCREEN_HEIGHT 64
15 #define OLED_RESET -1
16 #define SCREEN_ADDRESS 0x3C
17
18 #define I2C_SDA 21
19 #define I2C_SCL 22
20
21 Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
22
23 // ===== WiFi =====
24 const char* ssid = " ";
25 const char* senha = " ";
26
27 // ===== MQTT =====
28 const char* mqtt_server = "test.mosquitto.org";
29 const int mqtt_port = 1883;
30 const char* topico_ph = "aquario/ph";
31 const char* SERVIDOR_URL = "http://172.20.10.4:8000/api/ph"; // IP da máquina rodando
    o Docker
32
33 WiFiClient espClient;
34 PubSubClient client(espClient);
35
36 // ===== Calibração do pH =====
37 // pH = PH_SLOPE * tensao + PH_OFFSET
38 float PH_SLOPE = -5.70;
39 float PH_OFFSET = 21.34;
40
41 int buffer_arr[NUM_SAMPLES];
```

CÓDIGO

- Função: conectarWiFi()
- Mecanismo de Conexão;
- Feedback Visual e Tolerância a Falhas

```
1 void conectarWiFi()
2 {
3     Serial.print("Conectando ao WiFi");
4     display.clearDisplay();
5     display.setTextSize(1);
6     display.setCursor(0, 0);
7     display.println("Conectando ao WiFi");
8     display.println(ssid);
9     display.display();
10
11     WiFi.begin(ssid, senha);
12
13     int tentativas = 0;
14     while (WiFi.status() != WL_CONNECTED && tentativas < 40) // ~20s de timeout
15     {
16         delay(500);
17         Serial.print(".");
18         display.print(".");
19         display.display();
20         tentativas++;
21     }
22
23     display.clearDisplay();
24     display.setCursor(0, 0);
25
26     if (WiFi.status() == WL_CONNECTED)
27     {
28         Serial.println("\nConectado!");
29         Serial.print("IP: ");
30         Serial.println(WiFi.localIP());
31
32         display.println("WiFi conectado!");
33         display.println(WiFi.localIP());
34     }
35     else
36     {
37         Serial.println("\nFalha ao conectar ao WiFi.");
38         Serial.println("Continuando sem WiFi...");
39
40         display.println("Falha no WiFi.");
41         display.println("Continuando offline");
42     }
43
44     display.display();
45     delay(1500);
46 }
```

CÓDIGO

- Função: reconectarMQTT()
- Lógica de Funcionamento;
- Resiliência de Rede;

```
1 void reconectarMQTT()  
2 {  
3     if (WiFi.status() != WL_CONNECTED) return; // sem WiFi não adianta tentar  
4  
5     while (!client.connected())  
6     {  
7         Serial.print("Conectando ao broker MQTT ... ");  
8         String clientId = "ESP32-pH-" + String(random(0xffff), HEX);  
9         if (client.connect(clientId.c_str()))  
10        {  
11            Serial.println("conectado!");  
12        }  
13        else  
14        {  
15            Serial.print("falhou, rc=");  
16            Serial.print(client.state());  
17            Serial.println(" tentando de novo em 2s");  
18            delay(2000);  
19        }  
20    }  
21 }
```

CÓDIGO

- Função: setup()
- Etapas de Inicialização:
 - Comunicação Serial;
 - Ajuste do ADC;
 - Display OLED;
 - Rede e Serviços.

```
1 void setup()
2 {
3     Serial.begin(115200);
4     delay(1000);
5
6     Serial.println();
7     Serial.println("=====");
8     Serial.println("  MONITOR DE PH - PH-4502C");
9     Serial.println("=====");
10
11     analogReadResolution(12);
12     analogSetPinAttenuation(PH_PIN, ADC_11db);
13
14     // Inicializa I2C nos pinos padrão da ESP32
15     Wire.begin(I2C_SDA, I2C_SCL);
16
17     if (!display.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADDRESS))
18     {
19         Serial.println("Falha ao iniciar o display OLED!");
20         while (1) delay(10);
21     }
22
23     display.clearDisplay();
24     display.setTextColor(SSD1306_WHITE);
25     display.setTextSize(1);
26     display.setCursor(0, 0);
27     display.println("Monitor de pH");
28     display.println("Iniciando ... ");
29     display.display();
30
31     delay(1500);
32
33     WiFi.mode(WIFI_STA); // evita instabilidade em hardware real
34     conectarWiFi();
35
36     client.setServer(mqtt_server, mqtt_port);
37
38     Serial.println("Iniciando leitura ... \n");
39 }
```

CÓDIGO

- Função: lerTensaoFiltrada()
- Algoritmo de Filtragem de Ruído;
- Cálculo da Tensão;

```
1  float lerTensaoFiltrada()
2  {
3      for (int i = 0; i < NUM_SAMPLES; i++)
4      {
5          buffer_arr[i] = analogRead(PH_PIN);
6          delay(20);
7      }
8
9      for (int i = 0; i < NUM_SAMPLES - 1; i++)
10         for (int j = i + 1; j < NUM_SAMPLES; j++)
11             if (buffer_arr[i] > buffer_arr[j])
12             {
13                 int temp      = buffer_arr[i];
14                 buffer_arr[i] = buffer_arr[j];
15                 buffer_arr[j] = temp;
16             }
17
18         long soma = 0;
19         int inicio = NUM_SAMPLES / 4;
20         int fim     = NUM_SAMPLES - inicio;
21         int qtd     = fim - inicio;
22
23         for (int i = inicio; i < fim; i++)
24             soma += buffer_arr[i];
25
26         float leituraMedia = (float)soma / qtd;
27         return (leituraMedia * 3.3) / 4095.0;
28     }
```

CÓDIGO

- Função: mostrarNoOLED()
- Informações Apresentadas no Ecrã:
 - Status de Conetividade;
 - Leitura Elétrica;
 - Valor Medido.

```
1 void mostrarNoOLED(float tensao, float ph)
2 {
3     display.clearDisplay();
4
5     display.setTextSize(1);
6     display.setCursor(0, 0);
7     display.print("Monitor de pH ");
8     display.println(WiFi.status() == WL_CONNECTED ? "\n[WiFi On]" : "\n[WiFi Off]");
9     display.drawLine(0, 10, SCREEN_WIDTH, 10, SSD1306_WHITE);
10
11    display.setCursor(0, 18);
12    display.print("Tensao: ");
13    display.print(tensao, 3);
14    display.println(" V");
15
16    display.setTextSize(2);
17    display.setCursor(0, 34);
18    display.print("pH ");
19    display.print(ph, 2);
20
21    display.display();
22 }
```

CÓDIGO

- Integração Cloud / IoT:
- publicarMQTT():
 - Monta a payload JSON.
- enviarLeitura():
 - Envia um pedido HTTP POST com cabeçalho.

```
1 void publicarMQTT(float tensao, float ph)
2 {
3     String payload = "{\"ph\":\"" + String(ph, 2) + "\",\"tensao\":\"" + String(tensao, 3) + "\"}";
4     client.publish(topico_ph, payload.c_str());
5     Serial.println("Publicado no MQTT: " + payload);
6 }
7
8 void enviarLeitura(float ph, float tensao)
9 {
10     if (WiFi.status() != WL_CONNECTED)
11         return;
12
13     HTTPClient http;
14     http.begin(SERVIDOR_URL);
15     http.addHeader("Content-Type", "application/json");
16
17     String payload = "{\"ph\":\"" + String(ph, 2) +
18                     "\",\"voltage\":\"" + String(tensao, 3) +
19                     "\",\"device\":\"esp32-ph01\"}";
20
21     int codigo = http.POST(payload);
22
23     Serial.printf("POST enviado, resposta HTTP: %d\\n", codigo);
24
25     http.end();
26 }
```

CÓDIGO

- Função: loop()
- Fluxo Contínuo de Operação:
 - Autocura de Conexão;
 - Aquisição;
 - Output Local;
 - Telemetria;
 - Periodicidade;

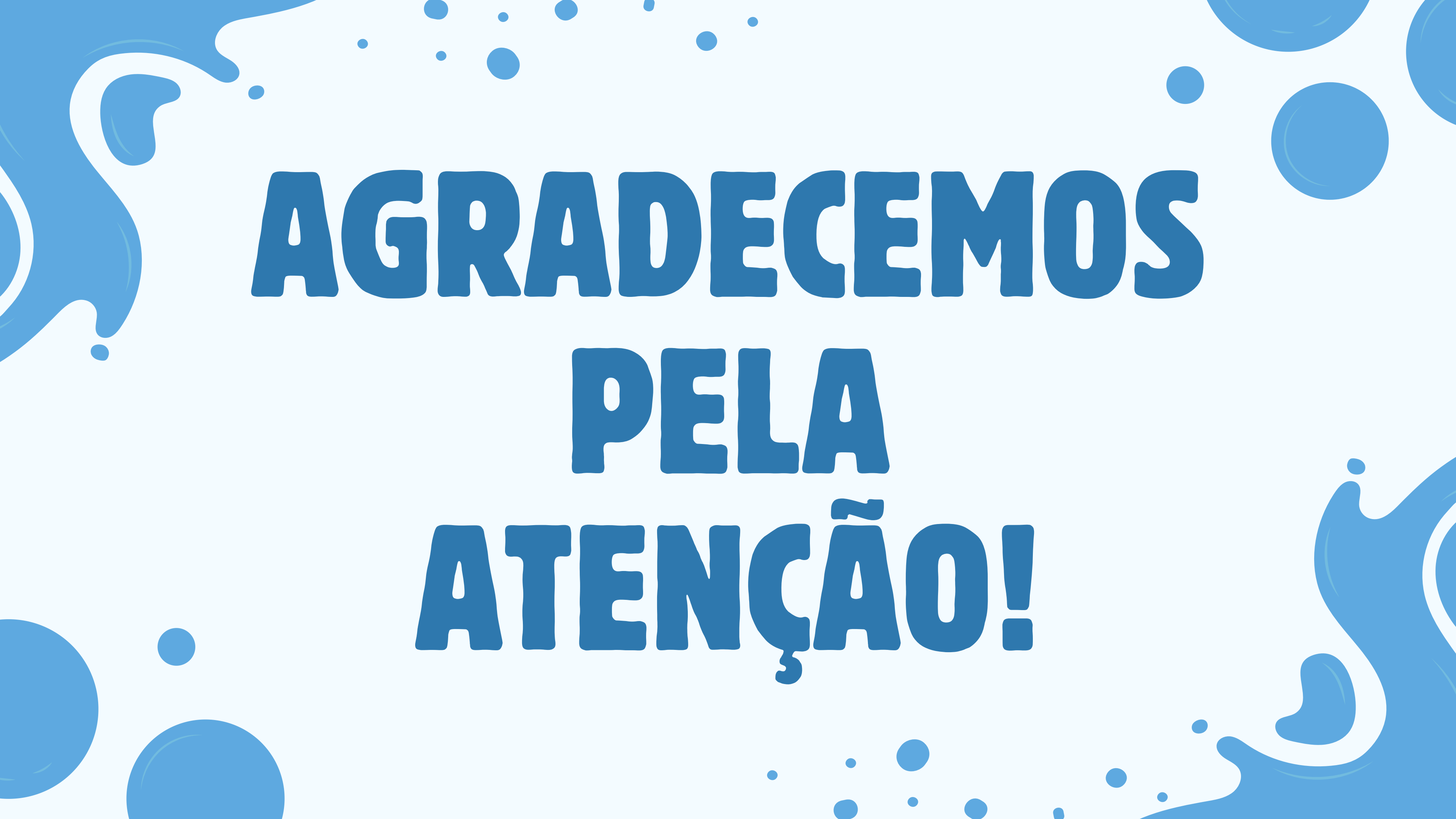
```
1 void loop()
2 {
3     // Tenta reconectar automaticamente caso a conexão caia
4     if (WiFi.status() != WL_CONNECTED)
5     {
6         Serial.println("WiFi desconectado. Tentando reconectar ...");
7         conectarWiFi();
8     }
9
10    reconectarMQTT();
11    client.loop();
12
13    float tensao = lerTensaoFiltrada();
14    float ph = PH_SLOPE * tensao + PH_OFFSET;
15
16    Serial.print("Tensao: ");
17    Serial.print(tensao, 3);
18    Serial.print(" V | pH: ");
19    Serial.println(ph, 2);
20
21    enviarLeitura(ph, tensao);
22    mostrarNoOLED(tensao, ph);
23    publicarMQTT(tensao, ph);
24
25    delay(1000);
26 }
```

EXPLICAÇÃO

- **Includes & Definições:**
 - WiFi.h, PubSubClient.h, HTTPClient.h: comunicação sem fios, MQTT e requisições Web HTTP.
 - Adafruit_SSD1306.h + Wire.h: controlo do ecrã OLED via barramento I2C.
 - Configuração de pinos (PH_PIN 35) e constantes de calibração do sensor (PH_SLOPE, PH_OFFSET).
- **Conetividade (conectarWiFi / reconectarMQTT):**
 - Tenta ligar ao Wi-Fi com timeout de ~20s; se falhar, continua a funcionar offline.
 - Garante reconexão automática ao Broker MQTT gerando um ID de cliente único.
- **setup():**
 - Inicia a comunicação Serial a 115200 bps e ajusta o ADC para 12 bits (~3.3V).
 - **Inicializa o ecrã OLED, estabelece a ligação à rede e configura o servidor MQTT.**

EXPLICAÇÃO

- **Processamento (lerTensaoFiltrada):**
 - Coleta 15 amostras e aplica ordenação (Bubble Sort) para eliminar picos de ruído elétrico.
 - Calcula a média dos valores centrais (Trimmed Mean) e converte para Tensão em Volts.
- **Saída & Nuvem (mostrarNoOLED, publicarMQTT, enviarLeitura):**
 - Atualiza o ecrã com estado da rede, tensão e o valor do pH em fonte grande.
 - Monta a payload JSON e envia as leituras via protocolo MQTT e requisição HTTP POST para a API/Servidor.
- **loop():**
 - Executa o ciclo contínuo a cada 1 segundo (delay(1000)).
 - Valida conexões, lê o sensor filtrado, calcula $\text{pH} = \text{SLOPE} \times \text{Tensão} + \text{OFFSET}$, atualiza a tela e envia os dados para a nuvem.



**AGRADECEMOS
PELA
ATENÇÃO!**